

Choose an execution scenario

Follow these graphical paths to see how selected build tools and runtime functions lead to the next step. Each box links to its captured behavior and exact source anchors; implementation and compiler PDFs provide the deeper listings.

These are curated scenarios reconstructed from source, not recorded dynamic traces or exact compiler control-flow graphs. Conditional branches, subprocess phases, asynchronous completions, and normal returns have distinct edge labels. Other platform paths and failure paths may exist.

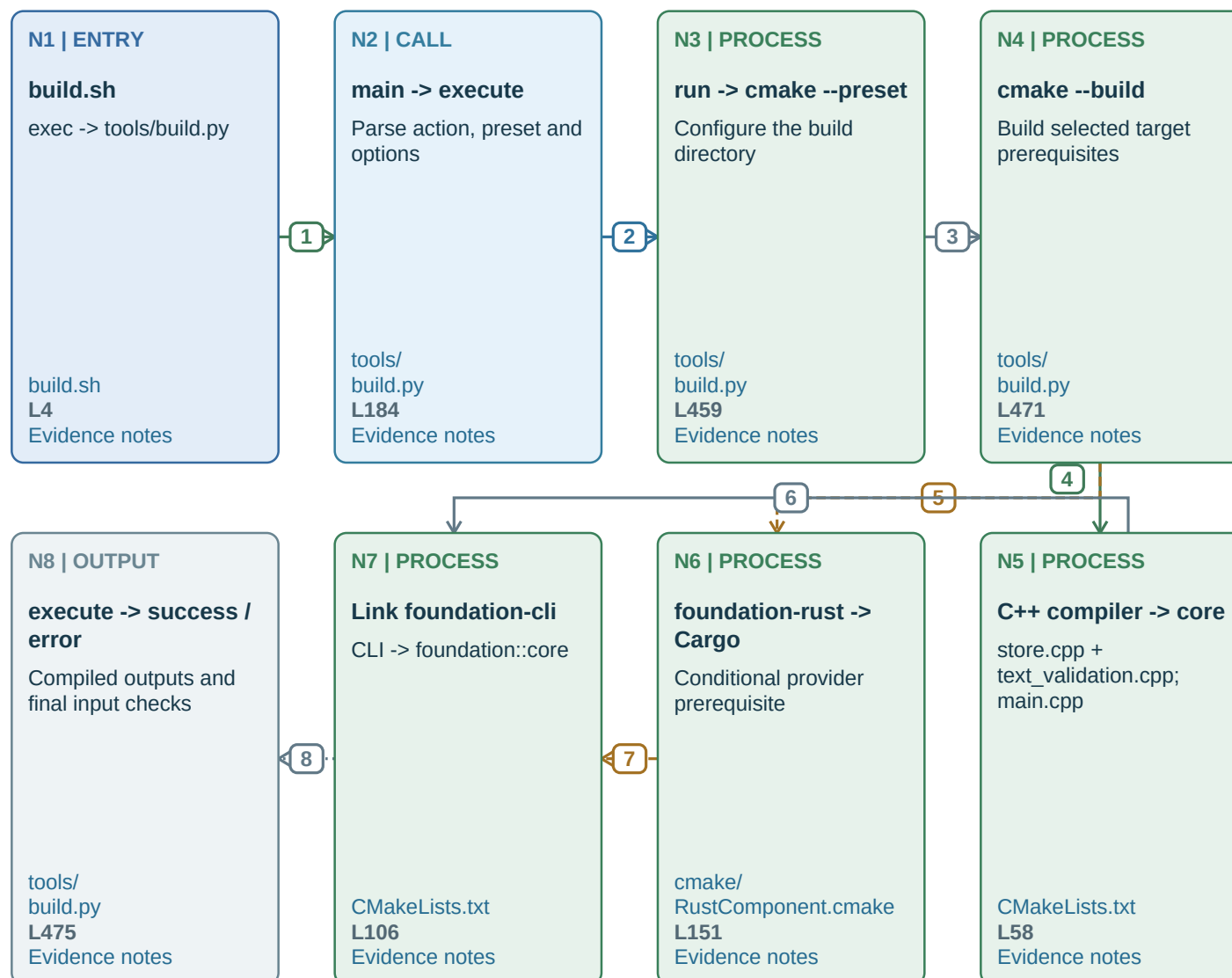
Snapshot: 2026-10-04T23:16:35-05:00. Generated only on explicit request; this navigation snapshot may be stale.

Execution flowchart	Question / scope	Captured steps
Compile the application	What code and tools are called when I ask to compile?	8
Run selected tests	What code is called when I ask to test, and where does execution enter a test?	8
Package, publish and promote a release	Which code creates archives, and which code changes a remote release or Latest?	8
First GUI run: entry point to event loop	What executes when the GUI starts, before the first user action?	8
Button click: Add entry to visible result	Which functions are called by the existing Add button?	8
Import: menu request to atomic replacement	What executes after Actions -> Import entries?	8
Export: captured records to host handoff	What executes after Actions -> Export entries?	8

[Graphical edit paths](#) | [Implementation walkthroughs](#) | [Compiler reference](#) | [Interactive execution chooser](#)

Compile the application

What code and tools are called when I ask to compile?



Directed edge key

- 1. N1 -> N2** [subprocess/command]
exec Python
- 2. N2 -> N3** [call]
execute -> timing -> run
- 3. N3 -> N4** [local step/order]
back in execute: --build
- 4. N4 -> N5** [subprocess/command]
generated compiler steps
- 5. N4 -> N6** [condition]
if provider = rust
- 6. N5 -> N7** [local step/order]
generator: objects ready
- 7. N6 -> N7** [condition]
if Rust: link prerequisite
- 8. N7 -> N8** [return]
process result + identity

CALL means a literal function call; PROCESS means an executable or generated build step; STEP means ordered orchestration. The Rust branch is conditional. Parallel target prerequisites are not a fixed chronological trace.

ENTRY CALL PROCESS DECISION EVENT OUTPUT

Click a box for its code diagram

Captured behavior behind this scenario

Representative build route: build.sh build dev enters the Python wrapper, configures CMake and builds selected targets. The CLI/core path is shown, with a conditional Rust component; optional GUI and other targets follow the configured graph. CMake source declarations describe inputs, while the selected generator actually schedules compiler/linker processes. The diagram does not resolve presets, variables, conditions or generator expressions.

[Up to flowchart](#) | [Scenario in explorer](#)

N1 | ENTRY | build.sh

Captured behavior: The shell wrapper finds its directory and replaces itself with the selected Python interpreter, passing the requested action, preset and remaining arguments to tools/build.py.

Path context: This is the user entry point, and exec is a process handoff.

build.sh:4

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [compile-entry](#)

N2 | CALL | main -> execute

Captured behavior: main parses the CLI, creates PhaseTimings, then calls execute. execute selects build/dependency/provider inputs and rejects incompatible or changed configuration identities before invoking build tools.

Path context: Arguments and retained input identity determine the configured path.

[tools/build.py:184](#) | [Interactive function/type](#)

main calls execute: [tools/build.py:229](#) | [Interactive function/type](#)

Build operation implementation: [tools/build.py:236](#) | [Interactive function/type](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [compile-main](#)

N3 | PROCESS | run -> cmake --preset

Captured behavior: execute constructs the configure command and calls PhaseTimings.call("configure", run, ...). run delegates to windows_compiler.run, which launches the child process using the selected platform wrapper.

Path context: Configuration produces the target graph; reading CMake alone does not configure it.

[tools/build.py:459](#) | [Interactive function/type](#)

Timing wrapper calls the supplied function: [tools/build.py:145](#) | [Interactive function/type](#)

Build helper calls the platform process wrapper: [tools/build.py:134](#) | [Interactive function/type](#)

Non-Windows child process invocation: [tools/windows_compiler.py:156](#) | [Interactive function/type](#)

Symbolic configure command construction: [tools/build.py:291](#) | [Interactive function/type](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [compile-configure](#)

N4 | PROCESS | cmake --build

Captured behavior: The same process wrapper runs cmake --build with the chosen build directory, concurrency and targets. A plain build selects all. The configured generator schedules each target and its prerequisites.

Path context: Compiler commands and their ordering depend on configuration and generator.

[tools/build.py:471](#) | [Interactive function/type](#)

Plain-build default target selection: [tools/build.py:461](#) | [Interactive function/type](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [compile-build](#)

N5 | PROCESS | C++ compiler -> core

Captured behavior: The generated build compiles foundation_core from the declared store and text-validation C++ files, and foundation-cli from main.cpp. The Rust provider still retains this C++ Store/front-end layer.

Path context: These are declarative source inputs to generated compiler processes.

[CMakeLists.txt:58](#) | [Read in PDF, p4](#)

Declared CLI source file: [CMakeLists.txt:105](#) | [Read in PDF, p4](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [compile-cpp](#)

N6 | PROCESS | foundation-rust -> Cargo

Captured behavior: When the Rust provider is selected, the custom target invokes rust_build.py build and verify. The builder runs Cargo build --frozen with the selected target/configuration and one job, then the configured Rust static component becomes a core link prerequisite.

Path context: C++ provider selection omits this branch. Cargo build and Cargo test are distinct operations.

[cmake/RustComponent.cmake:151](#) | [Interactive function/type](#)

CMake custom Rust archive verification command:

[cmake/RustComponent.cmake:152](#) | [Interactive function/type](#)

Rust builder creates the Cargo command: [tools/rust_build.py:629](#) | [Interactive function/type](#)

Selected Rust component linked into the C++ core:

[cmake/RustComponent.cmake:173](#) | [Interactive function/type](#)

Default provider selection: [CMakeLists.txt:31](#) | [Read in PDF](#), p20

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [compile-rust](#)

N7 | PROCESS | Link foundation-cli

Captured behavior: The generated linker step combines the CLI object with foundation::core and its selected dependencies. Optional GUI executable targets have their own configured source and link declarations.

Path context: Both relevant C++ objects and any selected Rust prerequisite must be ready; their work may overlap.

[CMakeLists.txt:106](#)

Selected Rust component linked into the C++ core:

[cmake/RustComponent.cmake:173](#) | [Interactive function/type](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [compile-link](#)

N8 | OUTPUT | execute -> success / error

Captured behavior: After compilation, the wrapper rechecks source identity and selected SDK/tool inputs. Only an unchanged, successful operation returns zero; child-process and identity failures produce an error exit.

Path context: A compiler-produced binary alone does not establish a stable input identity.

[tools/build.py:475](#) | [Interactive function/type](#)

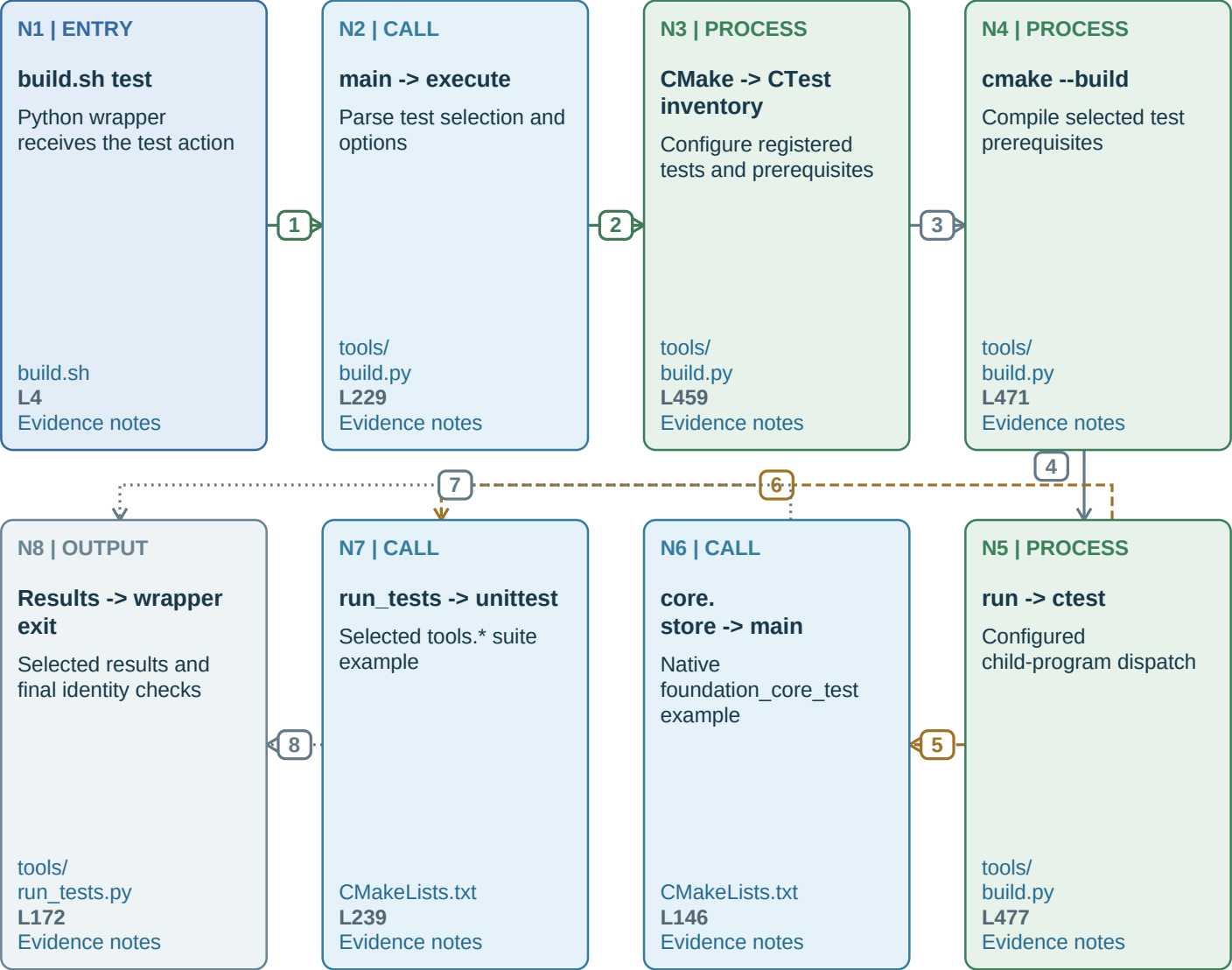
Selected SDK identity checked before success: [tools/build.py:526](#) | [Interactive function/type](#)

Build CLI failure exit: [tools/build.py:548](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [compile-result](#)

What code is called when I ask to test, and where does execution enter a test?



Directed edge key

- 1. N1 -> N2 [subprocess/command]
exec Python
- 2. N2 -> N3 [subprocess/command]
execute -> configure
- 3. N3 -> N4 [local step/order]
execute: build prerequisites
- 4. N4 -> N5 [local step/order]
back in execute: CTest
- 5. N5 -> N6 [condition]
if native core.store selected
- 6. N5 -> N7 [condition]
if tools.* suite selected
- 7. N6 -> N8 [return]
program -> CTest result
- 8. N7 -> N8 [return]
suite -> CTest result

CALL means a source-visible call;
PROCESS means a launched program;
STEP means ordered orchestration.
Conditional branches are representative
CTest dispatch choices, not an assertion
that these tests always execute.

Captured behavior behind this scenario

Representative test route: build.sh test dev configures and compiles the selected registered prerequisites before invoking CTest. The two child branches below illustrate core.store and tools.*; they are not the entire test inventory. Labels, named tests, full selection and platform conditions determine which programs actually run. This documentation only reads declarations and never invokes the test runner.

[Up to flowchart](#) | [Scenario in explorer](#)

N1 | ENTRY | build.sh test

Captured behavior: The shell entry passes test, preset and selection arguments into tools/build.py. An explicit test action is an application-tool invocation; the documentation generator never calls it.

Path context: The same wrapper owns both compilation prerequisites and test dispatch.

build.sh:4

Build CLI parses arguments: [tools/build.py:184](#) | [Interactive function/type](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [test-entry](#)

N2 | CALL | main -> execute

Captured behavior: main calls execute. The action, --tests or --label/--full choice, test concurrency and optional JUnit output are parsed before configuration and selection.

Path context: Selected tests and configured platform decide the runtime branch.

[tools/build.py:229](#) | [Interactive function/type](#)

Named test prerequisite selection: [tools/build.py:466](#) | [Interactive function/type](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [test-main](#)

N3 | PROCESS | CMake -> CTest inventory

Captured behavior: The configure subprocess evaluates add_test declarations and registered prerequisite targets. foundation_finalize_test_prerequisites writes the configured inventory and creates aggregate foundation-tests targets.

Path context: The documentation shows declarations; the real configured inventory is produced by CMake.

[tools/build.py:459](#) | [Interactive function/type](#)

Registered prerequisite metadata: [cmake/TestPrerequisites.cmake:6](#) | [Interactive function/type](#)

Aggregate test prerequisite target: [cmake/TestPrerequisites.cmake:80](#) | [Interactive function/type](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [test-configure](#)

N4 | PROCESS | cmake --build

Captured behavior: Named test selection supplies its registered prerequisite targets; label/full selection uses foundation-tests or its label-specific aggregate. The wrapper builds them before starting CTest.

Path context: CTest does not implicitly compile an EXCLUDE_FROM_ALL test executable.

[tools/build.py:471](#) | [Interactive function/type](#)

Label/full aggregate selection: [tools/build.py:469](#) | [Interactive function/type](#)

Core test executable is excluded from plain all: [CMakeLists.txt:142](#) | [Read in PDF, p5](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [test-build](#)

N5 | PROCESS | run -> ctest

Captured behavior: execute constructs ctest --test-dir ... --output-on-failure --no-tests=error and applies selection/concurrency. PhaseTimings calls run to launch it. Optional --show-only discovery is a timing probe, not test execution.

Path context: CTest uses the configured test commands, including native, Python and conditional Rust/GUI programs.

[tools/build.py:477](#) | [Interactive function/type](#)

CTest execution subprocess: [tools/build.py:499](#) | [Interactive function/type](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [test-ctest](#)

N6 | CALL | core.store -> main

Captured behavior: For a selected native core.store test, CTest launches foundation_core_test, entering tests/store_test.cpp main. It calls Store operations and checks accepted/rejected input and identity behavior. The browser configuration instead wraps the compiled test with its retained Node runtime.

Path context: This is an actual test-program entry point, not the application CLI main.

CMakeLists.txt:146

Core contract test entry: tests/store_test.cpp:26 | [Interactive function/type](#)

Core contract test calls Store: tests/store_test.cpp:31 | [Interactive function/type](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | test-core

N7 | CALL | run_tests -> unittest

Captured behavior: For a selected tools.* test, CTest launches run_tests.py with a registered suite. main loads that suite and calls execute; execute invokes unittest.TextTestRunner and records individual results and platform exclusions.

Path context: The selected suite is dynamic; incomplete required outcomes do not become a passing result.

CMakeLists.txt:239

Suite main calls execute: tools/run_tests.py:171 | [Interactive function/type](#)

unittest child calls: tools/run_tests.py:117 | [Interactive function/type](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | test-python

N8 | OUTPUT | Results -> wrapper exit

Captured behavior: Child programs return results to CTest; failed CTest execution fails the wrapper. Python suites publish their result file and return zero only for passed status. After execution, execute rechecks source and selected dependency/tool identity before reporting success.

Path context: A selected passing subset is not a claim of complete release certification.

tools/run_tests.py:172 | [Interactive function/type](#)

Python suite pass/fail exit: tools/run_tests.py:173 | [Interactive function/type](#)

Final source identity after additional writers: tools/build.py:519 | [Interactive function/type](#)

Selected SDK identity checked before success: tools/build.py:526 | [Interactive function/type](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | test-result

Which code creates archives, and which code changes a remote release or Latest?



Captured behavior behind this scenario

The ordinary hosted lifecycle below links separate workflow jobs/processes. The manual RELEASE procedure may stop at a complete local release and certify it locally first; delivered bytes must still be certified before promotion. Hosted production requires full regression before assembly/publication. Remote helpers default to planning (execute=False); an execute-enabled lifecycle is a distinct explicit choice. This diagram reads those routes only.

[Up to flowchart](#) | [Scenario in explorer](#)

N1 | DECISION | Manual / hosted release

Captured behavior: RELEASE documents manual local steps. The hosted ordinary lifecycle calls sdk-application with require_regression=true, then certification and promotion when execute is selected. Its receipt gate checks the nested full regression rather than rerunning it.

Path context: A local package request does not itself publish a candidate or change Latest.

[RELEASE:1](#)

Hosted application workflow handoff: [.github/workflows/_release-latest.yml:106](#)

Hosted full-regression requirement: [.github/workflows/_release-latest.yml:115](#)

[Open the code-level flowchart](#)

Captured source excerpt

```
1 Software Foundation \u2014 release and recovery order
2
3 1. Freeze source, version, target/backend inventory and
  qualification policy.
4 2. Verify supplier terms, pinned inputs and notices.
  Resolve GUI terms before
```

Print excerpt: 4 of 7 captured snippet lines; 3 omitted here. Follow the source/PDF links for surrounding implementation.

[Up to chart](#) | [Interactive scenario step](#) | [release-entry](#)

N2 | PROCESS | build.py -> CPack

Captured behavior: The package action configures/builds, then calls CPack with CPackConfig.cmake. Hosted prepared_package first runs build.py test release --full against prepared inputs, then changes that command to package. The selected archive is copied into the producer handoff.

Path context: CPack creates an archive; it neither assembles a complete release nor uploads it.

[tools/build.py:501](#) | [Interactive function/type](#)

Hosted producer starts with full test action: [tools/ci_plan.py:307](#) | [Interactive function/type](#)

Hosted producer switches to package action: [tools/ci_plan.py:345](#) | [Interactive function/type](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [release-package](#)

N3 | CALL | artifact.describe / verify

Captured behavior: The manual procedure calls artifact.py create and verify. Hosted prepared_package calls artifact.describe and artifact.verify on the selected archive. The build wrapper also offers these subprocesses through --verify-package; ordinary package creation alone does not run them.

Path context: These checks describe/verify archive contents and relocation, not certification or remote publication.

[tools/ci_plan.py:357](#) | [Interactive function/type](#)

Hosted actual archive verification: [tools/ci_plan.py:358](#) | [Interactive function/type](#)

Artifact creation CLI: [tools/artifact.py:314](#) | [Interactive function/type](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [release-inventory](#)

N4 | CALL | release.assemble

Captured behavior: ci_plan.assemble_release or the manual release CLI calls release.assemble. It copies the prebuilt source/archive inputs and exact dependency groups into new staging, verifies the complete release, then commits the new local directory. The hosted assembly job also requires its full regression result to succeed.

Path context: Assembly consumes existing archives. It neither recompiles them nor publishes remotely.

[tools/ci_plan.py:399](#) | [Interactive function/type](#)

Local assembly implementation: [tools/release.py:278](#) | [Interactive function/type](#)

Hosted publication waits for required regression:

[.github/workflows/sdk-application.yml:272](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | release-assemble

N5 | PROCESS | publish_candidate

Captured behavior: An execute-enabled publisher creates a new tag/draft, uploads immutable candidate assets, verifies the remote inventory/digests, then publishes it as a prerelease. execute=False returns a plan. The candidate deliberately remains outside Latest while certification is pending.

Path context: The remote helper is a separate operation after local assembly; existing tags/releases are not overwritten.

[tools/github_release.py:1097](#) | [Interactive function/type](#)

Candidate publication explicitly keeps Latest false: [tools/github_release.py:1117](#) | [Interactive function/type](#)

Hosted explicit publication call: [.github/scripts/lifecycle.py:933](#) | [Interactive function/type](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | release-publish

N6 | CALL | Qualify -> certify -> attach

Captured behavior: The hosted certify workflow qualifies exact downloaded candidate bytes in required environments. lifecycle calls qualification_tasks.prepare/run, then certify_release.certify merges the exact reports, checks inventory/policy/environment coverage and emits eligibility. attach_certificate appends that attempt.

Path context: A green packaging job alone is insufficient. Certification neither publishes nor promotes, and experiments are ineligible.

[.github/workflows/_release-latest.yml:121](#)

Physical qualification dispatch: [.github/scripts/lifecycle.py:955](#) | [Interactive function/type](#)

Certificate calculation call: [.github/scripts/lifecycle.py:966](#) | [Interactive function/type](#)

Certification eligibility rule: [tools/certify_release.py:99](#) | [Interactive function/type](#)

Certificate attachment implementation: [tools/github_release.py:1200](#) | [Interactive function/type](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | release-certificate

N7 | CALL | github_release.promote

Captured behavior: An explicit execute=True promotion requires the exact certification run, attempt and certificate SHA. It verifies the remote asset inventory and certificate, rejects experiments/ineligible evidence, then changes the ordinary release to stable Latest. The hosted metadata-only handoff uses verified API digests and bound evidence.

Path context: Changing Latest is a separate remote mutation. Helper defaults remain planning-only.

[tools/github_release.py:1281](#) | [Interactive function/type](#)

Exact remote certificate revalidation: [tools/github_release.py:1295](#) | [Interactive function/type](#)

Hosted promotion call and execute choice: [.github/scripts/lifecycle.py:991](#) | [Interactive function/type](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | release-promote

N8 | OUTPUT | Verify Latest identity

Captured behavior: promote rechecks the release lifecycle, assets, tag and /releases/latest pointer. The hosted final job independently calls latest_release.py verify and retains its receipt. A preparation-only run records that no public candidate/certification/Latest change was requested.

Path context: A successful HTTP mutation is followed by identity checks; a local release may stop before any remote step.

[tools/github_release.py:1304](#) | [Interactive function/type](#)

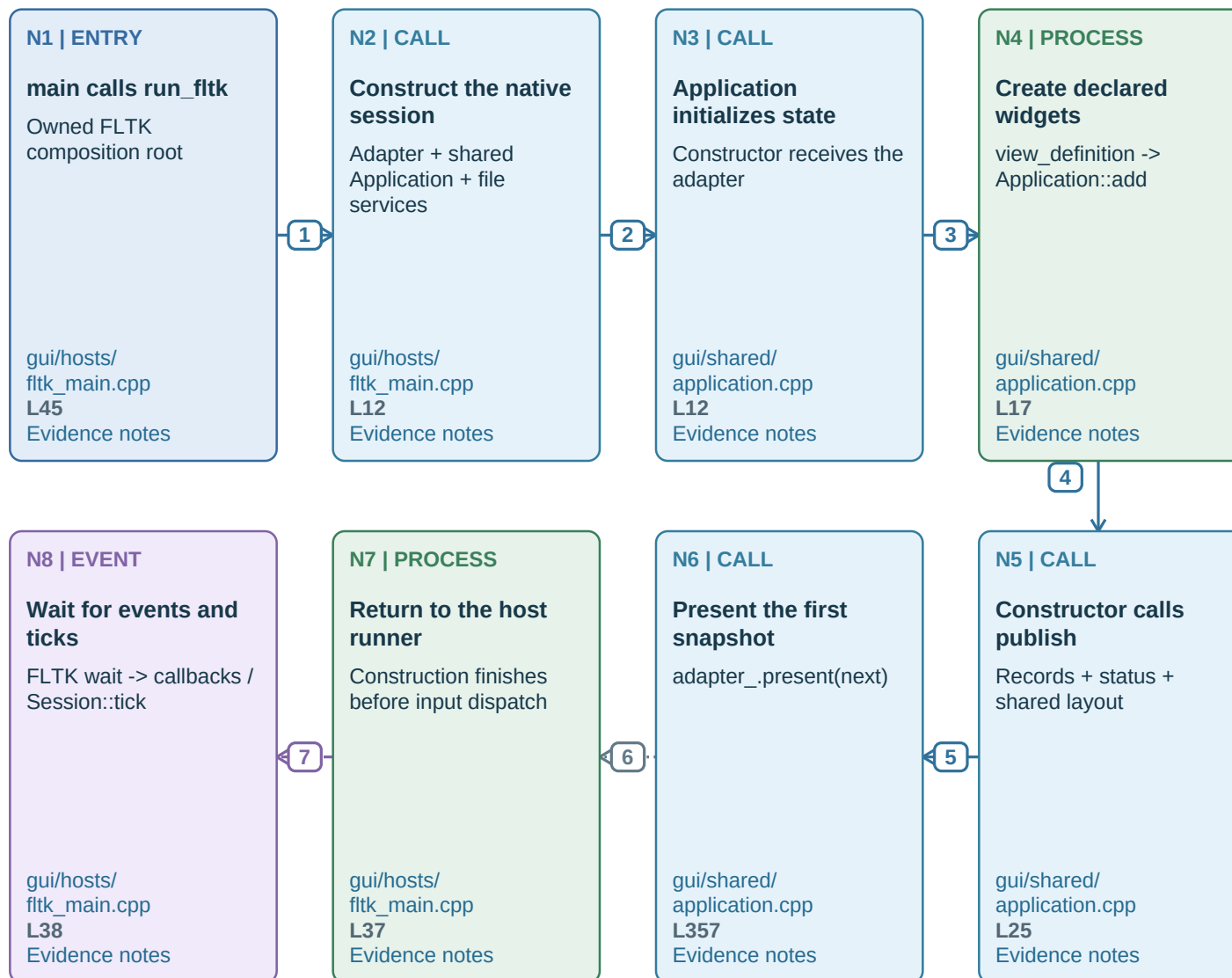
Hosted final independent verification: [.github/workflows/_release-latest.yml:167](#)

Manual separately authorized publication boundary: [RELEASE:33](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [release-result](#)

What executes when the GUI starts, before the first user action?



Directed edge key

1. N1 -> N2 [call]
run_fltk constructs
2. N2 -> N3 [call]
application(adapter)
3. N3 -> N4 [call]
add(definition) loop
4. N4 -> N5 [call]
publish()
5. N5 -> N6 [call]
adapter_.present(next)
6. N6 -> N7 [return]
successful construction
7. N7 -> N8 [event/completion]
show / timer / wait

These arrows describe existing execution. CALL is a named source call; PROCESS nodes group work and STEP arrows continue local statements. RETURN resumes a caller, including caught exception unwinding; EVENT crosses a callback, queue or asynchronous-delivery boundary. Conditional arrows name the selected branch. Supplier adapter internals are outside this walkthrough.

Captured behavior behind this scenario

This concrete path follows a normal FLTK launch without qualification flags. Other hosts supply different adapters but create the same shared Application. Browser startup references show the corresponding factory and receive boundary. The CLI is a separate entry in `src/main.cpp` and does not create this GUI.

[Up to flowchart](#) | [Scenario in explorer](#)

N1 | ENTRY | main calls `run_fltk`

Captured behavior: The executable main calls `run_fltk<foundation::ui::Application>(argc, argv)`. The runner checks its arguments before constructing the session.

Path context: This is one real GUI entry point, rather than a compiler or documentation entry.

[gui/hosts/ftlk_main.cpp:45](#) | [Interactive function/type](#)

Separate CLI entry logic: [src/main.cpp:13](#) | [Interactive function/type](#) | [Read in PDF, p83](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [start-entry](#)

N2 | CALL | Construct the native session

Captured behavior: `run_fltk` constructs `NativeSession<App, gui::ftlk::Adapter>`. `NativeSession` aliases `Session<App, Adapter, FileServices>`; `Session` constructs the adapter callback before `application(adapter)`. The callback starts forwarding only after `current_` is assigned.

Path context: The host chooses an adapter and generic services; it does not define product widgets.

[gui/hosts/ftlk_main.cpp:12](#) | [Interactive function/type](#)

Native session alias: [gui/host/native_session.hpp:7](#)

Application construction in Session: [gui/host/contract.hpp:47](#) | [Interactive function/type](#) | [Read in PDF, p22](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [start-session](#)

N3 | CALL | Application initializes state

Captured behavior: The shared constructor requires its selected task executor, sets the window title, and begins creating declared controls. `Browser<App>` also constructs this same App with its `WebAdapter`.

Path context: Application owns the Store and feature state, while the adapter remains host-supplied.

[gui/shared/application.cpp:12](#) | [Interactive function/type](#) | [Read in PDF, p20](#)

Browser creates the shared App: [gui/host/browser.hpp:19](#) | [Interactive function/type](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [start-application](#)

N4 | PROCESS | Create declared widgets

Captured behavior: The constructor loops over `view_definition` and calls `add` for rows outside the optional Remove extension. `add` copies identity, kind, text, label, font and wrapping into widgets and assigns the current root parent. The constructor then configures editor policy, menu options and list behavior.

Path context: The declaration table is consumed before the first snapshot is presented.

[gui/shared/application.cpp:17](#) | [Interactive function/type](#) | [Read in PDF, p20](#)

Widget creation method: [gui/shared/application.cpp:29](#) | [Interactive function/type](#) | [Read in PDF, p31](#)

Widget declaration table: [gui/shared/view_definition.hpp:18](#) | [Read in PDF, p119](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [start-widgets](#)

N5 | CALL | Constructor calls publish

Captured behavior: Before the constructor returns, `publish` stages next from `view_`, projects Store records and status, composes the column layout, copies bounds, and sets enabled flags.

Path context: Geometry and feature state are prepared by shared code before crossing the adapter boundary.

[gui/shared/application.cpp:25](#) | [Interactive function/type](#) | [Read in PDF, p20](#)

Shared publication body: [gui/shared/application.cpp:300](#) | [Interactive function/type](#) | [Read in PDF, p26](#)

Shared layout call: [gui/shared/application.cpp:333](#) | [Interactive function/type](#) | [Read in PDF, p26](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [start-layout](#)

N6 | CALL | Present the first snapshot

Captured behavior: publish increments the staged revision and calls `adapter_.present(next)`. After that call succeeds it commits next to `view_` and clears `presentation_pending_`. The concrete adapter body is outside the captured owned-source walkthrough.

Path context: A failed presentation does not turn this boundary into a successful return.

[gui/shared/application.cpp:357](#) | [Interactive function/type](#) | [Read in PDF](#), p26

Commit after presentation: [gui/shared/application.cpp:358](#) | [Interactive function/type](#) | [Read in PDF](#), p26

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [start-present](#)

N7 | PROCESS | Return to the host runner

Captured behavior: Successful publish returns through Application construction and Session construction to `run_fltk`. The runner shows the window and schedules its timer. Browser entry points instead construct `Browser<App>` and obtain its initial transport response.

Path context: Native and browser startup share Application construction but use different outer event loops.

[gui/hosts/ftlk_main.cpp:37](#) | [Interactive function/type](#)

Hosted browser composition: [gui/hosts/web_main.cpp:46](#) | [Interactive function/type](#)

Wasm creation entry: [gui/hosts/wasm_main.cpp:13](#) | [Interactive function/type](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [start-return](#)

N8 | EVENT | Wait for events and ticks

Captured behavior: The normal native runner waits while the adapter is open. Its timer calls `Session::tick`, which calls `Application::tick` and pumps services. Adapter callbacks call `Application::handle`. Browser receives and ordered polls reach the same shared event/tick logic through Browser and WebSession.

Path context: Startup hands control to ongoing input and progress delivery; these are later invocations.

[gui/hosts/ftlk_main.cpp:38](#) | [Interactive function/type](#)

Native timer calls session tick: [gui/hosts/ftlk_main.cpp:32](#) | [Interactive function/type](#)

Session tick boundary: [gui/host/contract.hpp:63](#) | [Interactive function/type](#)

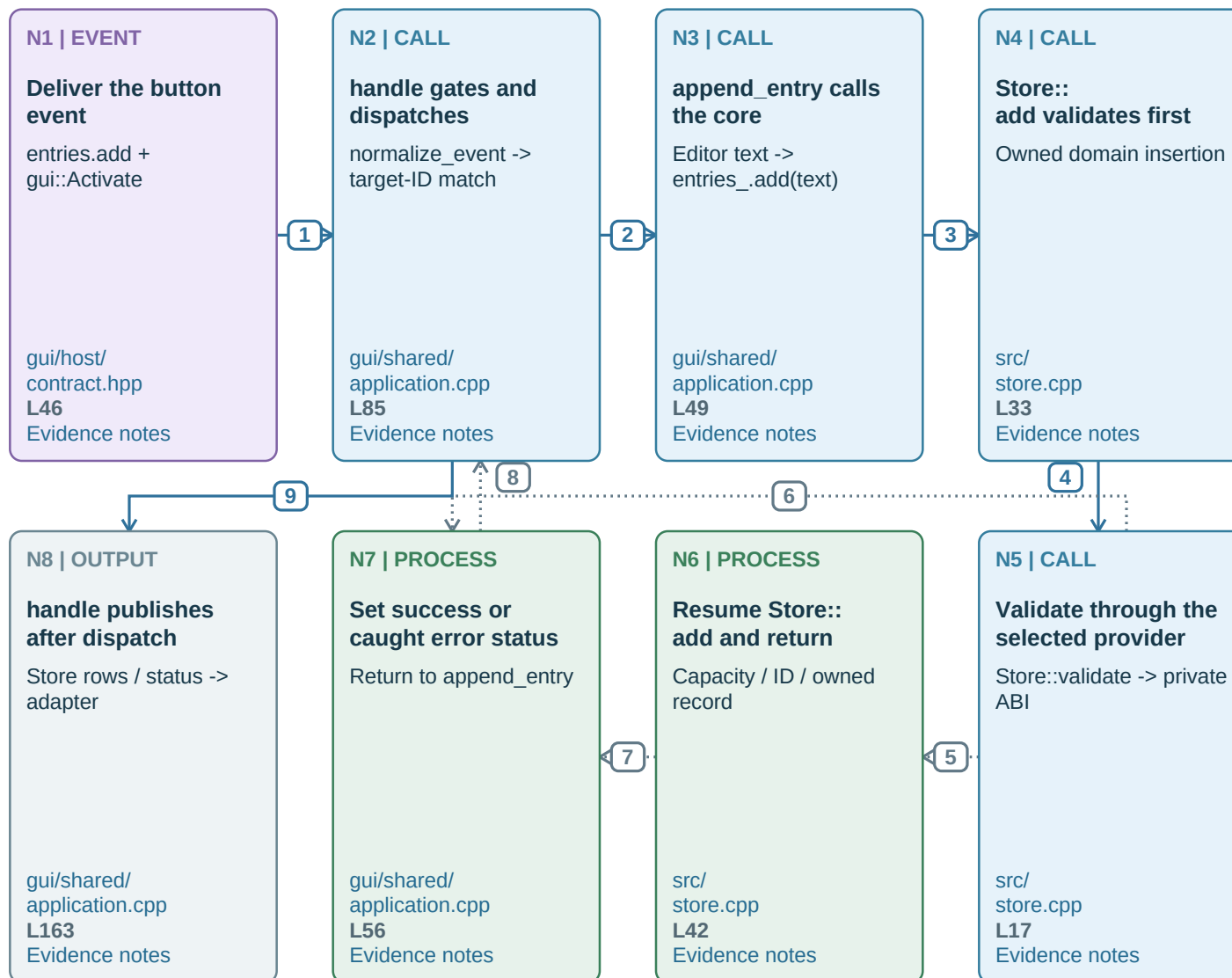
Native event callback: [gui/host/contract.hpp:46](#) | [Interactive function/type](#) | [Read in PDF](#), p22

Browser receive boundary: [gui/host/browser.hpp:26](#) | [Interactive function/type](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [start-loop](#)

Which functions are called by the existing Add button?



Directed edge key

1. N1 -> N2 [call]
handle(event)
2. N2 -> N3 [call]
Activate + entries.add
3. N3 -> N4 [call]
entries_.add(text)
4. N4 -> N5 [call]
validate(text)
5. N5 -> N6 [return]
validation succeeded
6. N5 -> N7 [return]
throws invalid_argument
7. N6 -> N7 [return]
ID / caught length_error
8. N7 -> N2 [return]
visitor resumes
9. N2 -> N8 [call]
after normal dispatch

These arrows describe existing execution. CALL is a named source call; PROCESS nodes group work and STEP arrows continue local statements. RETURN resumes a caller, including caught exception unwinding; EVENT crosses a callback, queue or asynchronous-delivery boundary. Conditional arrows name the selected branch. Supplier adapter internals are outside this walkthrough.

Captured behavior behind this scenario

This follows `entries.add` with `Activate`; editor `SubmitText` also calls `append_entry`. The native callback is shown concretely and browser forwarding is linked separately. Only caught input/capacity errors return through this normal publication path; other exceptions may escape.

[Up to flowchart](#) | [Scenario in explorer](#)

N1 | EVENT | Deliver the button event

Captured behavior: A supported adapter supplies `WidgetEvent` for the declared button. The native `Session` callback calls the current `Application::handle(event)`; the browser callback uses its `application_` pointer.

Path context: The callback-to-application calls are owned source; vendor input translation is not expanded here.

[gui/host/contract.hpp:46](#) | [Interactive function/type](#) | [Read in PDF](#), p22

Existing Add button: [gui/shared/view_definition.hpp:22](#) | [Read in PDF](#), p119

Browser event callback: [gui/host/browser.hpp:15](#) | [Interactive function/type](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [click-delivery](#)

N2 | CALL | handle gates and dispatches

Captured behavior: `handle` rejects closed/shut-down state and normalizes against authoritative `view_` and scroll offsets. Rejected input returns without publication. For accepted `Activate` on `entries.add` the visitor calls `append_entry`.

Path context: A delivered event must still pass current availability, identity and input policy.

[gui/shared/application.cpp:85](#) | [Interactive function/type](#) | [Read in PDF](#), p5

Existing Add dispatch: [gui/shared/application.cpp:101](#) | [Interactive function/type](#) | [Read in PDF](#), p5

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [click-handle](#)

N3 | CALL | append_entry calls the core

Captured behavior: `append_entry` advances `input_epoch` before mutation, reads `entries.editor` text and calls `entries_.add(text)` inside its try block. Epoch exhaustion occurs before that try block and can escape.

Path context: This helper owns application status and editor behavior, rather than widget dispatch.

[gui/shared/application.cpp:49](#) | [Interactive function/type](#) | [Read in PDF](#), p11

Core insertion call: [gui/shared/application.cpp:54](#) | [Interactive function/type](#) | [Read in PDF](#), p11

Authoritative semantic epoch: [gui/shared/application.cpp:67](#) | [Interactive function/type](#) | [Read in PDF](#), p87

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [click-append](#)

N4 | CALL | Store::add validates first

Captured behavior: `Store::add` calls `validate(text)` before checking capacity, identifier availability and insertion. It owns records independently of the GUI.

Path context: No list-row mutation can replace this core operation.

[src/store.cpp:33](#) | [Interactive function/type](#) | [Read in PDF](#), p23

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [click-core](#)

N5 | CALL | Validate through the selected provider

Captured behavior: `Store::validate` calls `detail::validate_text`, which calls `foundation_text_validate_v1`. The configured Rust or C++ provider returns a numeric status; `Store` translates it to success or an exception.

Path context: Provider choice is conditional configuration, not two calls made for each click.

[src/store.cpp:17](#) | [Interactive function/type](#) | [Read in PDF](#), p72

Private validation wrapper: [src/text_validation.cpp:32](#) | [Interactive function/type](#) | [Read in PDF](#), p116

Rust provider entry: [rust/text_validation/src/lib.rs:28](#) | [Interactive function/type](#) | [Read in PDF](#), p117

Selected C++ provider entry: [src/text_validation.cpp:14](#) | [Interactive function/type](#) | [Read in PDF](#), p76

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [click-validator](#)

N6 | PROCESS | Resume Store::add and return

Captured behavior: After successful validation, add rejects a full collection or exhausted ID, then pushes an owned record and increments `next_id_`. Its `RecordId` returns to `append_entry`; this caller does not use the returned ID.

Path context: Validation returns into `Store::add`. It does not call GUI publication.

[src/store.cpp:42](#) | [Interactive function/type](#) | [Read in PDF](#), p23

Collection capacity failure: [src/store.cpp:36](#) | [Interactive function/type](#) | [Read in PDF](#), p23

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [click-insert](#)

N7 | PROCESS | Set success or caught error status

Captured behavior: Success clears editor text and sets Entry added. Caught `invalid_argument` or `length_error` sets error status without clearing the editor. Other exceptions escape. The helper itself never calls `publish`.

Path context: Status handling returns to `handle`; publication is the caller's subsequent action.

[gui/shared/application.cpp:56](#) | [Interactive function/type](#) | [Read in PDF](#), p11

Caught input error: [gui/shared/application.cpp:58](#) | [Interactive function/type](#) | [Read in PDF](#), p11

Caught capacity error: [gui/shared/application.cpp:61](#) | [Interactive function/type](#) | [Read in PDF](#), p11

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [click-status](#)

N8 | OUTPUT | handle publishes after dispatch

Captured behavior: After the visitor returns normally, `handle` calls `publish`. Shared projection rebuilds list rows, status, layout and enabled state before calling `adapter_.present(next)`. Presentation failure leaves `retry` debt.

Path context: The visible result follows normal dispatch completion, not a direct call from validation.

[gui/shared/application.cpp:163](#) | [Interactive function/type](#) | [Read in PDF](#), p5

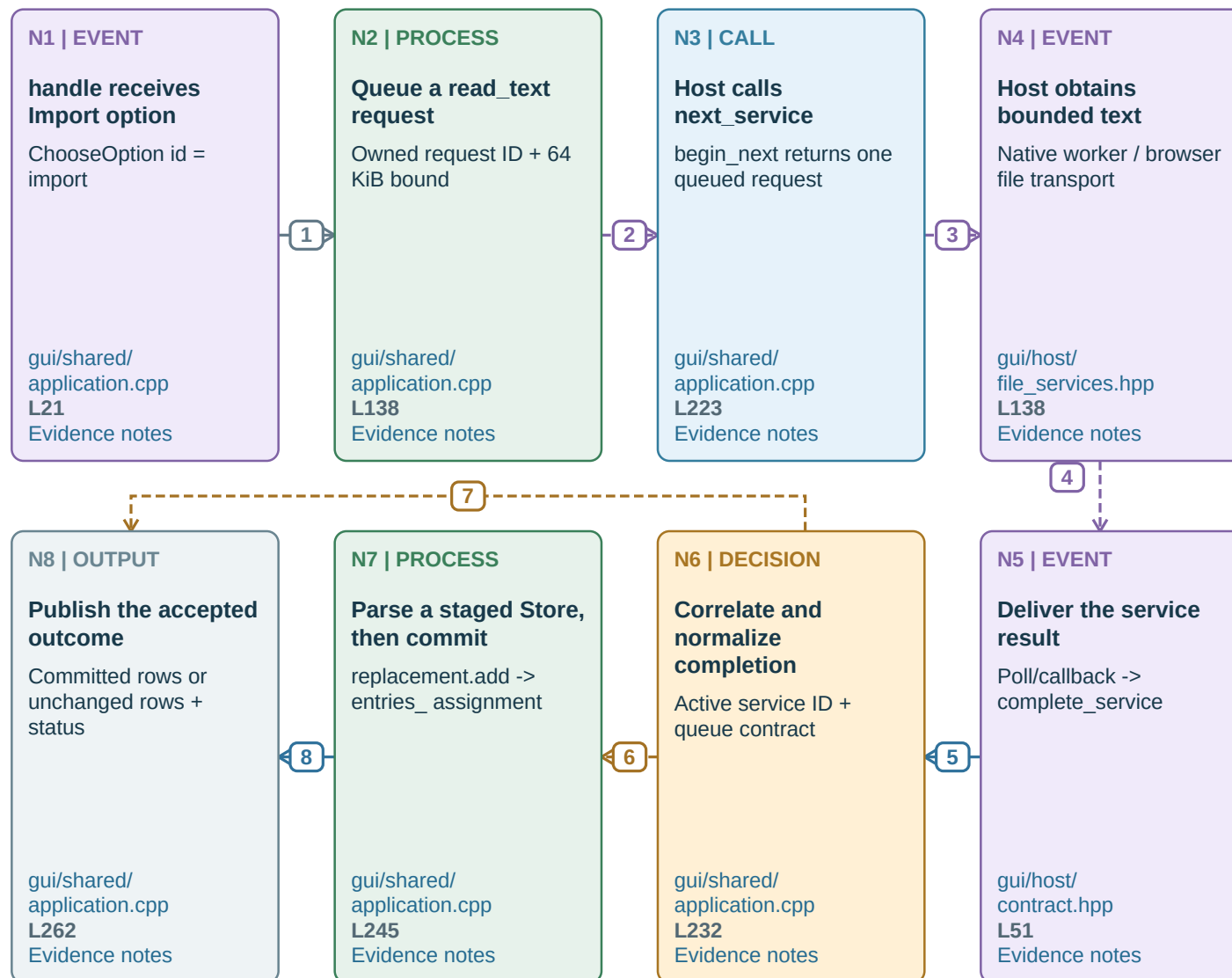
Shared publication body: [gui/shared/application.cpp:300](#) | [Interactive function/type](#) | [Read in PDF](#), p26

Generic adapter presentation call: [gui/shared/application.cpp:357](#) | [Interactive function/type](#) | [Read in PDF](#), p26

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [click-publish](#)

What executes after Actions -> Import entries?



Directed edge key

1. N1 -> N2 [local step/order]
ChooseOption branch
2. N2 -> N3 [event/completion]
after handle returns: host pump
3. N3 -> N4 [event/completion]
dispatch content capability
4. N4 -> N5 [event/completion]
asynchronous result
5. N5 -> N6 [call]
complete_service(result)
6. N6 -> N7 [condition]
successful read_text
7. N6 -> N8 [condition]
accepted cancellation / error
8. N7 -> N8 [call]
commit or caught parse error

These arrows describe existing execution. CALL is a named source call; PROCESS nodes group work and STEP arrows continue local statements. RETURN resumes a caller, including caught exception unwinding; EVENT crosses a callback, queue or asynchronous-delivery boundary. Conditional arrows name the selected branch. Supplier adapter internals are outside this walkthrough.

Captured behavior behind this scenario

Shared code requests owned UTF-8 text with a 64 KiB maximum. Native selectors/worker and browser file transport differ, then deliver the same correlated ServiceResult. Parsing mutates a staged Store and commits the complete replacement only on success.

[Up to flowchart](#) | [Scenario in explorer](#)

N1 | EVENT | handle receives Import option

Captured behavior: The menu event reaches handle through the host callback and passes `normalize_event` before its `ChooseOption` branch recognizes import.

Path context: Import begins with shared menu intent, not an application file-path call.

[gui/shared/application.cpp:21](#) | [Interactive function/type](#) | [Read in PDF](#), p20

Service command route: [gui/shared/application.cpp:133](#) | [Interactive function/type](#) | [Read in PDF](#), p5

Application event normalization: [gui/shared/application.cpp:85](#) | [Interactive function/type](#) | [Read in PDF](#), p5

Native event callback: [gui/host/contract.hpp:46](#) | [Interactive function/type](#) | [Read in PDF](#), p22

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [import-action](#)

N2 | PROCESS | Queue a read_text request

Captured behavior: handle assigns the next service ID, `read_text` kind and title, with `byte_limit` 65536. It advances `input_epoch`, enqueues the request and updates pending counters. Normal handle dispatch then publishes before returning to the native service pump or browser bridge.

Path context: Enqueueing does not itself read a file or call `next_service`.

[gui/shared/application.cpp:138](#) | [Interactive function/type](#) | [Read in PDF](#), p5

Shared request byte limit: [gui/shared/application.cpp:141](#) | [Interactive function/type](#) | [Read in PDF](#), p5

Queue the owned service request: [gui/shared/application.cpp:151](#) | [Interactive function/type](#) | [Read in PDF](#), p5

Authoritative semantic epoch: [gui/shared/application.cpp:67](#) | [Interactive function/type](#) | [Read in PDF](#), p87

handle publishes after normal dispatch: [gui/shared/application.cpp:163](#) | [Interactive function/type](#) | [Read in PDF](#), p5

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [import-request](#)

N3 | CALL | Host calls next_service

Captured behavior: The native Session services method checks host activity, calls `application.next_service` and passes the returned request to FileServices. `next_service` advances `input_epoch`, begins the queued request and decrements pending count. Browser WebSession uses the supplied `next_service` callback.

Path context: Queued and active requests are separate states; only one active request is yielded.

[gui/shared/application.cpp:223](#) | [Interactive function/type](#)

Begin the next queued service: [gui/shared/application.cpp:226](#) | [Interactive function/type](#)

Session calls the content provider: [gui/host/contract.hpp:58](#) | [Interactive function/type](#)

Browser request callback: [gui/host/browser.hpp:16](#) | [Interactive function/type](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [import-begin](#)

N4 | EVENT | Host obtains bounded text

Captured behavior: Native FileServices asks for a path on the host side, starts its worker and calls `file_detail::transfer` to read a regular file within the byte bound. Browser file helpers use fatal UTF-8 decoding and ordered upload/finish operations. No path or browser File enters Application.

Path context: This asynchronous boundary is transport-specific; record parsing remains shared.

[gui/host/file_services.hpp:138](#) | [Interactive function/type](#)

Native file worker starts: [gui/host/file_services.hpp:148](#) | [Interactive function/type](#)

Native bounded transfer: [gui/host/file_services.hpp:66](#) | [Interactive function/type](#) | [Read in PDF](#), p109

Browser text read: [gui/host/file_services.mjs:20](#) | [Interactive function/type](#) | [Read in PDF](#), p114

Browser UTF-8 decoding: [gui/host/file_services.mjs:29](#) | [Interactive function/type](#) | [Read in PDF](#), p114

Owned text-service contract extension: [gui/patches/file-services.patch:19](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [import-read](#)

N5 | EVENT | Deliver the service result

Captured behavior: Native Session polls the worker result on the UI owner and calls `complete_service`. Selector cancellation or errors may use its reply callback instead. Browser WebSession delivers through the supplied `complete_service` callback after its transport checks.

Path context: The file worker does not call Application or manipulate widgets.

[gui/host/contract.hpp:51](#) | [Interactive function/type](#)

Native immediate reply callback: [gui/host/contract.hpp:59](#) | [Interactive function/type](#)

Browser bridge calls shared completion: [gui/host/browser.hpp:17](#) | [Interactive function/type](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [import-delivery](#)

N6 | DECISION | Correlate and normalize completion

Captured behavior: `complete_service` rejects closed/shut-down state and a missing or mismatched active request. It advances `input_epoch` and calls `services_.complete(result)`, which consumes and normalizes the accepted result. Only successful `read_text` reaches parsing; cancellation or error updates status instead.

Path context: Unmatched results return false without parsing or publication; UTF-8/limit rejection becomes an error outcome.

[gui/shared/application.cpp:232](#) | [Interactive function/type](#) | [Read in PDF](#), p104

Match the currently active service ID: [gui/shared/application.cpp:235](#) | [Interactive function/type](#) | [Read in PDF](#), p104

Complete and normalize the queued result: [gui/shared/application.cpp:237](#) | [Interactive function/type](#) | [Read in PDF](#), p104

Transport error becomes shared status: [gui/shared/application.cpp:239](#) | [Interactive function/type](#) | [Read in PDF](#), p104

Cancellation becomes shared status: [gui/shared/application.cpp:240](#) | [Interactive function/type](#) | [Read in PDF](#), p104

Owned text-service contract extension: [gui/patches/file-services.patch:19](#)

[Open the code-level flowchart](#)

Captured source excerpt

```
231
232 bool Application::complete_service(gui::ServiceResult
    result) {
233     if (shutdown_ || adapter_.closed()) return false;
234     const auto request = services_.current();
```

Print excerpt: 4 of 8 captured snippet lines; 4 omitted here. Follow the source/PDF links for surrounding implementation.

[Up to chart](#) | [Interactive scenario step](#) | [import-accept](#)

N7 | PROCESS | Parse a staged Store, then commit

Captured behavior: The success branch copies entries_, erases old records in that copy, splits lines, strips a trailing CR and calls replacement.add for each record. Only after all records validate does it move replacement into entries_ and clear selection. An empty file replaces with an empty collection; malformed content leaves live records and next-ID state unchanged and sets error status.

Path context: Copying the existing Store preserves monotonic ID allocation; rebuilding a fresh Store would reuse IDs.

[gui/shared/application.cpp:245](#) | [Interactive function/type](#) | [Read in PDF, p104](#)

Validate each imported line: [gui/shared/application.cpp:253](#) | [Interactive function/type](#) | [Read in PDF, p104](#)

Commit imported records: [gui/shared/application.cpp:257](#) | [Interactive function/type](#) | [Read in PDF, p104](#)

Core add validates before insertion: [src/store.cpp:33](#) | [Interactive function/type](#) | [Read in PDF, p23](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [import-commit](#)

N8 | OUTPUT | Publish the accepted outcome

Captured behavior: Accepted success, parse failure, transport error or cancellation reaches publish from complete_service. It projects the resulting Store and shared status to the adapter, then completion returns true. A publication exception can escape while accepted authoritative changes remain.

Path context: The outcome distinguishes content acceptance from service delivery and painting.

[gui/shared/application.cpp:262](#) | [Interactive function/type](#) | [Read in PDF, p104](#)

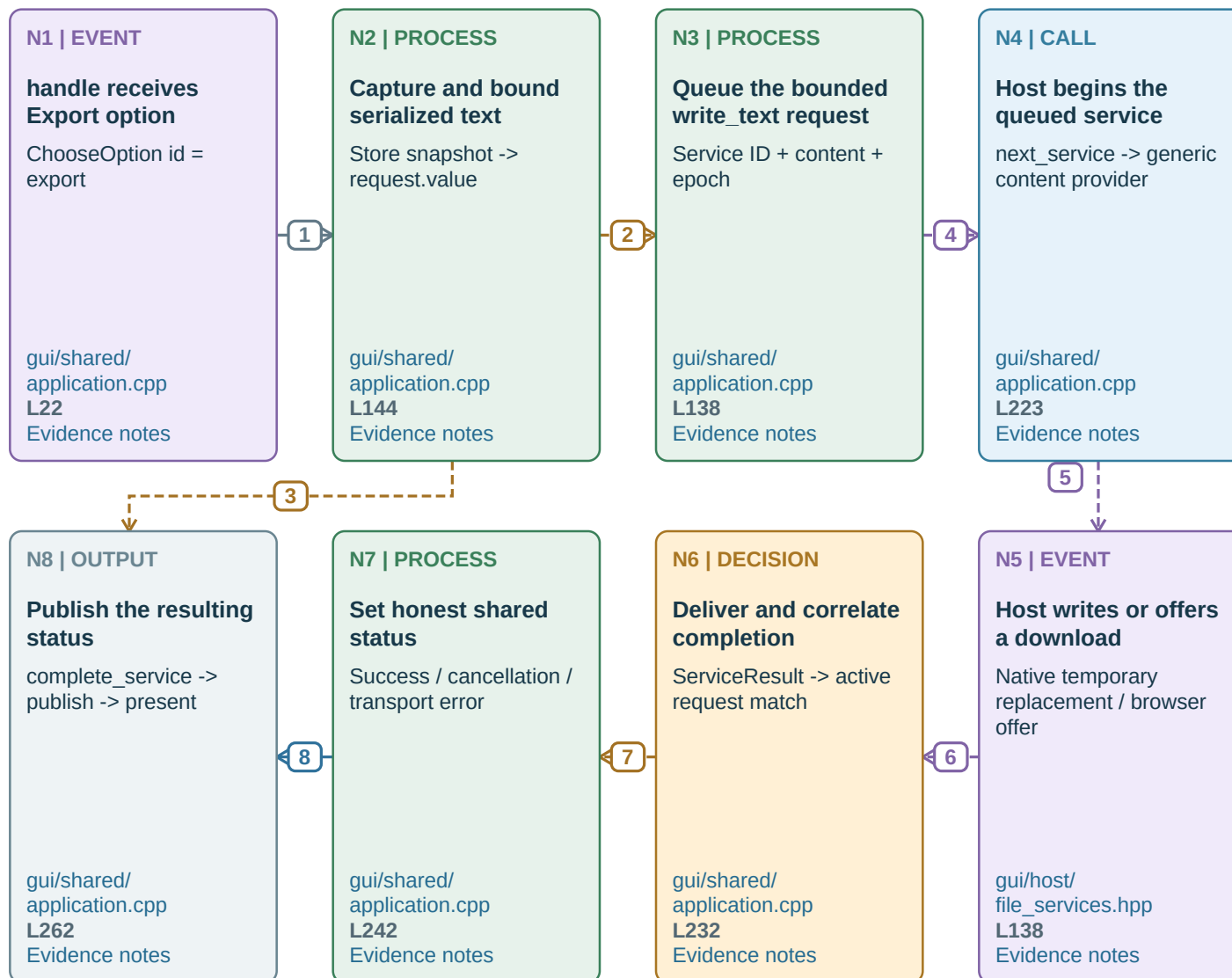
Shared publication body: [gui/shared/application.cpp:300](#) | [Interactive function/type](#) | [Read in PDF, p26](#)

Generic adapter presentation call: [gui/shared/application.cpp:357](#) | [Interactive function/type](#) | [Read in PDF, p26](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [import-publish](#)

What executes after Actions -> Export entries?



Directed edge key

1. N1 -> N2 [local step/order]
ChooseOption branch
2. N2 -> N3 [condition]
serialized bytes fit
3. N2 -> N8 [condition]
too large: local status
4. N3 -> N4 [event/completion]
after handle returns: host pump
5. N4 -> N5 [event/completion]
dispatch content capability
6. N5 -> N6 [event/completion]
asynchronous result / callback
7. N6 -> N7 [condition]
accepted outcome
8. N7 -> N8 [call]
publish()

These arrows describe existing execution. CALL is a named source call; PROCESS nodes group work and STEP arrows continue local statements. RETURN resumes a caller, including caught exception unwinding; EVENT crosses a callback, queue or asynchronous-delivery boundary. Conditional arrows name the selected branch. Supplier adapter internals are outside this walkthrough.

Captured behavior behind this scenario

Export captures record text when the command is accepted and sends owned UTF-8 content. Native transfer replaces a chosen destination after writing a temporary file; browser transfer offers a download. Shared success says Export handed to host, not durably saved.

[Up to flowchart](#) | [Scenario in explorer](#)

N1 | EVENT | handle receives Export option

Captured behavior: The declared menu event passes handle normalization and enters the export branch of shared service-command routing.

Path context: The feature request begins in shared code before any host file selector.

[gui/shared/application.cpp:22](#) | [Interactive function/type](#) | [Read in PDF](#), p20

Export serialization branch: [gui/shared/application.cpp:143](#) | [Interactive function/type](#) | [Read in PDF](#), p5

Application event normalization: [gui/shared/application.cpp:85](#) | [Interactive function/type](#) | [Read in PDF](#), p5

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [export-action](#)

N2 | PROCESS | Capture and bound serialized text

Captured behavior: The command serializes each Store record as text plus newline into owned request.value. It checks the complete byte count against byte_limit. Oversized output advances input_epoch, sets error status and returns from the visitor without enqueueing; handle still performs its normal publication.

Path context: Later record edits do not rewrite content already captured in the request.

[gui/shared/application.cpp:144](#) | [Interactive function/type](#) | [Read in PDF](#), p5

Serialized byte count check: [gui/shared/application.cpp:145](#) | [Interactive function/type](#) | [Read in PDF](#), p5

Shared request byte limit: [gui/shared/application.cpp:141](#) | [Interactive function/type](#) | [Read in PDF](#), p5

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [export-serialize](#)

N3 | PROCESS | Queue the bounded write_text request

Captured behavior: For output within the bound, handle uses write_text, advances input_epoch, enqueues its owned request and updates counters. It publishes and returns before host service delivery.

Path context: No file has been written by the enqueue call.

[gui/shared/application.cpp:138](#) | [Interactive function/type](#) | [Read in PDF](#), p5

Queue the owned service request: [gui/shared/application.cpp:151](#) | [Interactive function/type](#) | [Read in PDF](#), p5

Authoritative semantic epoch: [gui/shared/application.cpp:67](#) | [Interactive function/type](#) | [Read in PDF](#), p87

handle publishes after normal dispatch: [gui/shared/application.cpp:163](#) | [Interactive function/type](#) | [Read in PDF](#), p5

Owned text-service contract extension: [gui/patches/file-services.patch:19](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [export-request](#)

N4 | CALL | Host begins the queued service

Captured behavior: The service pump calls next_service, which begins one active request and advances input_epoch. Native Session passes it to FileServices; Browser supplies equivalent request/completion callbacks to WebSession.

Path context: Application passes owned content and receives a correlated result, not a host pathname.

[gui/shared/application.cpp:223](#) | [Interactive function/type](#)

Begin the next queued service: [gui/shared/application.cpp:226](#) | [Interactive function/type](#)

Session calls the content provider: [gui/host/contract.hpp:58](#) | [Interactive function/type](#)

Browser bridge calls shared completion: [gui/host/browser.hpp:17](#) | [Interactive function/type](#)

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | [export-begin](#)

N5 | EVENT | Host writes or offers a download

Captured behavior: Native FileServices obtains its selected path and its worker calls `file_detail::transfer`. That routine writes an exclusive sibling temporary file and replaces the destination only after completed writing and close when not cancelled. Browser file helpers retrieve bounded export content and offer a download. Cancellation before native commit preserves the destination; cancellation after commit cannot undo it.

Path context: These generic transport steps do not promise crash durability or that a user retained a browser download.

[gui/host/file_services.hpp:138](#) | Interactive function/type

Native file worker starts: [gui/host/file_services.hpp:148](#) | Interactive function/type

Native bounded transfer: [gui/host/file_services.hpp:66](#) | Interactive function/type | [Read in PDF, p109](#)

Native successful replacement marker: [gui/host/file_services.hpp:114](#) | Interactive function/type | [Read in PDF, p109](#)

Browser export reads: [gui/host/file_services.mjs:70](#) | Interactive function/type

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | export-write

N6 | DECISION | Deliver and correlate completion

Captured behavior: The native UI pump/reply callback or browser completion callback calls `complete_service`. Closed/shut-down or unmatched results return false. A matching active result advances `input_epoch` and passes through `services_.complete` before shared outcome handling.

Path context: A late or repeated result does not authorize another output request or shared success status.

[gui/shared/application.cpp:232](#) | Interactive function/type | [Read in PDF, p104](#)

Match the currently active service ID: [gui/shared/application.cpp:235](#) | Interactive function/type | [Read in PDF, p104](#)

Complete and normalize the queued result: [gui/shared/application.cpp:237](#) | Interactive function/type | [Read in PDF, p104](#)

Native session delivers a polled result: [gui/host/contract.hpp:51](#) | Interactive function/type

Browser bridge calls shared completion: [gui/host/browser.hpp:17](#) | Interactive function/type

[Open the code-level flowchart](#)

Captured source excerpt

```
231
232  bool Application::complete_service(gui::ServiceResult
    result) {
233      if (shutdown_ || adapter_.closed()) return false;
234      const auto request = services_.current();
```

Print excerpt: 4 of 8 captured snippet lines; 4 omitted here. Follow the source/PDF links for surrounding implementation.

[Up to chart](#) | [Interactive scenario step](#) | export-accept

N7 | PROCESS | Set honest shared status

Captured behavior: Successful write_text sets Export handed to host. Accepted cancellation sets Service cancelled; an error uses the normalized error text. This completion does not mutate Store records.

Path context: Success reports the content capability's handoff semantics, not durable storage.

[gui/shared/application.cpp:242](#) | [Interactive function/type](#) | [Read in PDF](#), p104

Transport error becomes shared status: [gui/shared/application.cpp:239](#) | [Interactive function/type](#) | [Read in PDF](#), p104

Cancellation becomes shared status: [gui/shared/application.cpp:240](#) | [Interactive function/type](#) | [Read in PDF](#), p104

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | export-status

N8 | OUTPUT | Publish the resulting status

Captured behavior: After accepted service outcome handling, complete_service calls publish and returns true on normal completion. The local oversized-output branch reaches publication directly from handle, without starting a service. Repainting does not serialize or write the export again.

Path context: Publication is a separate visible result after request rejection or service completion.

[gui/shared/application.cpp:262](#) | [Interactive function/type](#) | [Read in PDF](#), p104

handle publishes after normal dispatch: [gui/shared/application.cpp:163](#) | [Interactive function/type](#) | [Read in PDF](#), p5

Shared publication body: [gui/shared/application.cpp:300](#) | [Interactive function/type](#) | [Read in PDF](#), p26

Generic adapter presentation call: [gui/shared/application.cpp:357](#) | [Interactive function/type](#) | [Read in PDF](#), p26

[Open the code-level flowchart](#)

[Up to chart](#) | [Interactive scenario step](#) | export-publish